MindMemory

TECHNICAL BRIEF · FOR ENGINEERS AND RESEARCHERS

Memory for Claude Code.

A drop-in folder + Python hooks that give Claude Code a regenerable, markdown-native memory layer with continuous self-maintenance and a built-in measurement harness.

CoMindLab Labs · Research preview · MIT · April 2026

01 What it is

MindMemory is a portable memory layer for Claude Code. Three primitives:

- **Markdown files** in a structured layout — no vector DB, no service.
- **Five Python hooks** wired to Claude Code's PostToolUse, SessionStart, and SessionEnd events.
- **One setup wizard skill** that bootstraps a project with the right structure in ~5 minutes.

Total dependencies: zero. Standard library only. Read the source — it's the entire system.

Repository: github.com/CoMindLab-ai/mind-memory

02 Architecture (visual)

MindMemory Memory for Claude Code that gets better the more you use it.

PANEL A · CONCEPT

From agents to minds.

Capability has plateaued; **memory is the next lever**. A "mind" is identity + memory + heuristics, wired together as one markdown construct.

01

Agents

`stateless`

- High capability per call
- No memory
- No personality
- Transactional

02

Claude Code today

`session-scoped`

- High capability
- Session-only memory
- Thin prompted personality
- Transactional

MINDMEMORY

03

A mind

`colleague-like`

- High capability
- Persistent memory
- Stable identity
- Growing heuristics

04

Human mind

`aspiration`

- Variable capability
- Lifelong, selective
- Deep, stable
- Continuous learning

A mind is one markdown-based construct: `identity` + `memory` + `heuristics`. **Claude Code is the substrate. Markdown is the storage.** No vector DB, no fine-tuning.

PANEL B · MEMORY

Four tiers, by lifespan.

Organised by how long things live, not by topic. Index pattern keeps always-loaded context to ~1–2 KB while making thousands of memories available on demand.

B1 · The 4-tier stack

TOP = LONGER-LIVED

01 Identity — READ-ONLY
Personality. Rules it never breaks.
`identity/anchor.md`

02 Long-term memory — WEEKS+
Confirmed patterns, profile, lessons.
`memory/MEMORY.md · {topic}.md`

03 Working memory — DAYS
This week — focus, blockers, recent corrections.
`memory/working-memory.md`

04 Archive — RARELY READ
Compressed history. Not loaded by default.
`memory/archive.md`

B2 · Hunch → heuristic

3-CONFIRMATION RULE

1× **Observation** `logged`
Noted by collector. Not yet in memory.

2× **Hunch** `tentative`
Added to working-memory, labelled.

3× **Heuristic** `permanent`
Promoted to Tier 2. Applied going forward.

↺ **Contradicted** `demoted`
Returns to hunch, or archived with note.

Each Tier-2 entry stores **why** + **how to apply** — so the mind can judge edge cases instead of blindly following.

PANEL C · SELF-MAINTENANCE

Three concentric loops.

The mind curates itself **while you work**, not just at logoff. Catching corrections within minutes produces tighter rules than catching them hours later.

INNER LOOP — Mid-session — every N tool calls

PostToolUse hook

Collect signals — `memory-collector.py`
Every **~10 calls**: scans live transcript, appends to `working-memory.md` while context is hot.
- correction
- confirmation
- follow-through
- frustration

→

Promote / demote — `memory-curator.py`
Every **~25 calls**: applies the 3-confirmation rule. Hunch → heuristic. Contradicted → demoted. Stale (>5d) → archived.

MIDDLE LOOP — Per session — bookends

SessionStart · SessionEnd

Session start — `session-start-health.py`
Loads **identity** + **MEMORY.md** + **working-memory**. Warns if curator hasn't run in >7 days.

→

Session end — `session-metrics.py`
Writes one row to `session-metrics.csv`. Heavy maintenance already happened in the inner loop — this is the safety net.

OUTER LOOP — Weekly — time-gated

7-day timestamp

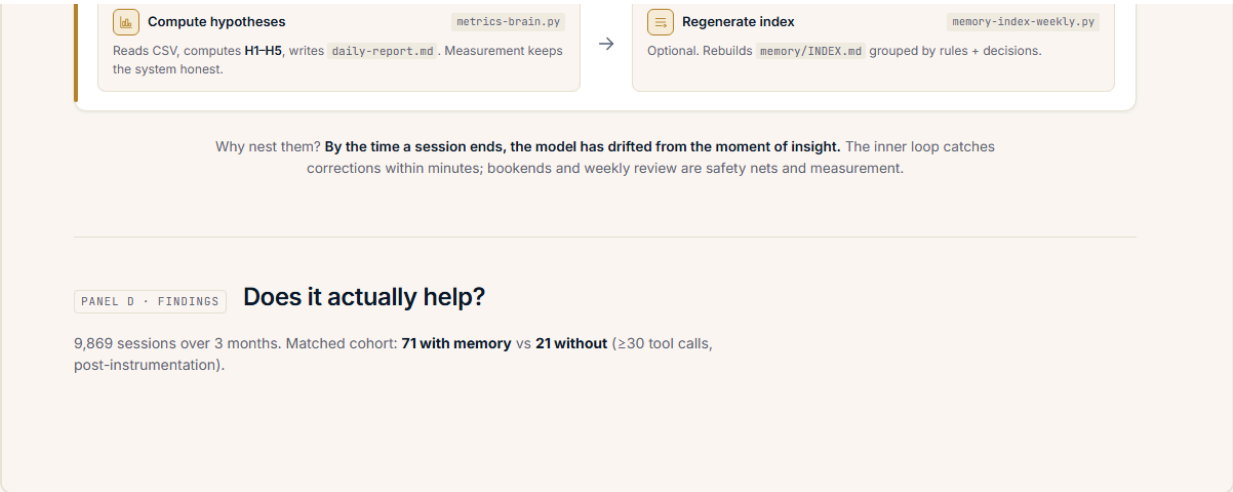

Figure 1. Single-page architecture reference. Source: MindMemory.html in the repo.

03 The four tiers

Memory is organised by **lifespan**, not by topic. This maps cleanly onto "what gets loaded at session start" vs "what gets retrieved on demand" vs "what stays out of context entirely."

Tier	File	Lifespan	Role
1 · Identity	`identity/anchor.md`	Read-only	Personality. Hard rules. Loaded every session.
2 · Long-term	`memory/MEMORY.md` + `{topic}.md`	Weeks+	Confirmed patterns. Indexed — index always loaded, details on demand.
3 · Working	`memory/working-memory.md`	Days	This week's focus. Token-budgeted (~5KB). Rolls into Tier 2 or 4.
4 · Archive	`memory/archive.md`	Rarely read	Compressed historical context. Searchable, not loaded by default.

The promotion rule. Borrowed from scientific reasoning to keep memory curated:

1. **1×** — observation, logged but not in memory
2. **2×** — hunch (tentative, labelled in working memory)
3. **3×** — heuristic (promoted to Tier 2, applied)
4. **Contradicted** — demoted to hunch or archived with note

Each Tier-2 entry stores both **why** (the reasoning behind the rule) and **how to apply** (concrete trigger). The "why" lets the model judge edge cases instead of blindly applying.

04 The three loops — continuous self-maintenance

Most "AI memory" systems consolidate at session end. MindMemory's defining design choice: **memory curates itself mid-session**, not just at logoff.

```
INNER LOOP   every ~10–25 tool calls (PostToolUse hook)
  memory-collector → scan transcript, capture signals
  memory-curator   → promote / demote per 3× rule

MIDDLE LOOP  per session
  SessionStart → load identity + MEMORY.md + working-memory
  SessionEnd   → write metrics row to CSV

OUTER LOOP   weekly (7-day timestamp gate)
  metrics-brain      → compute H1–H5, write daily-report.md
  memory-index       → regenerate INDEX.md (optional feature)
```

Why nest them? By the time a session ends, the model has drifted from the moment of insight. The inner loop catches corrections within minutes, producing tighter, more specific rules than catching them hours later. Bookends and weekly review are safety nets and measurement.

---

05 The signals collector + curator detect

Signal	Pattern	What it triggers
Correction	"no", "wrong", "don't do that"	Candidate new heuristic
Confirmation	"yes exactly", "perfect", "keep doing that"	Reinforce existing approach
Follow-through	User accepts output without changes	Silent confirmation
Frustration	"again?", "as I said", strong negative language	Escalate severity in working memory
Repeated topic	Same correction 2+ times in a session	Auto-promote to hunch

Pattern-matching only — no LLM call inside the collector. The curator does call `claude -p` for promotion judgment when a hunch hits 3× confirmation, because that's where world-knowledge actually helps.

---

06 Measurement — the H1–H5 hypotheses

Every install ships with a measurement harness. Five hypotheses, computed from the session-metrics CSV:

- **H1** — Production ratio: productive_calls / total_calls (↑ if memory helps)
- **H2** — Orientation: exploration calls before first edit (↓ if memory helps)
- **H3** — Agent efficiency: agent delegations / total calls (↓ if memory helps)
- **H4** — Density: productive calls per session (↑ if memory helps)
- **H5** — Repeat-correction rate: % sessions with repeated corrections (↓ if memory helps) — *primary outcome*

Run `python .claude/hooks/metrics-brain.py` after two weeks of use to see your own H1–H5. The system measures itself; users self-test whether memory is actually helping their work.

07 Findings — three months of data

Author dataset: 9,869 sessions over 12 weeks. Matched cohort (≥30 tool calls, post-instrumentation): 71 with memory loaded vs 21 without.

H1 PRODUCTION	H2 ORIENT	H4 DENSITY	H5 REPEAT
2.2×	−10%	4.3×	Null
18.5% vs 8.5%	14.6 vs 16.3	26.0 vs 6.0	19.7% vs 14.3%

What's robust: H1, H2, H4 are large effects, robust to a sensitivity check excluding the W15 merge window (where a parallel-implementation merge inflated repeat-corrections regardless of memory state).

What's null: H5 — the headline outcome — has not been demonstrated. Three honest theories in the findings doc:

- **Selection bias** — memory loads on harder sessions; the control group is biased toward easy work
- **Coverage gap** — memory doesn't have the right lessons yet
- **Recall gap** — memory exists but isn't applied under pressure

Cannot separate them with current n. Publishing the result is itself the contribution: **the system that measures itself is the only honest way to test "does memory in AI work?"**

> **The interesting bit for reviewers:** the qualitative report from the n=1 user ("you remembering things helps me") is consistent with H1/H2/H4 but invisible to H5. Format/pacing preferences (dyslexia, visual-learner) get used silently every session — no "wrong"/"again" language fires. The most valuable memory is the kind H5 can't detect.

08 Install — three commands

```
cp -r starter-kit/. your-project/
cd your-project
mv .claude/settings.json.example .claude/settings.json
claude
> /mind-memory-setup
```

The wizard asks 7 questions (with deep-dive opt-in for richer personality), respects existing files via an archive mechanism, never destructively modifies `CLAUDE.md` (uses a managed marker block), and writes to project-only / global / auto-split scope per your choice.

09 What we want from reviewers

1. **Critique the experimental design.** If H5 is the wrong primary outcome, tell us what's better. `04-open-questions.md` has the full list.
2. **Run it on your work, share the CSV.** One user's data is anecdotal. Twenty users' data starts to be evidence.
3. **Find counter-examples.** "I put this in memory and Claude still didn't apply it" is gold. Specific cases beat general theories.

10 Lineage and license

The kernel is Andrej Karpathy's LLM Wiki framing — an agent that writes down what worked and what didn't in structured natural language, then reads it back. Not fine-tuning, not vector search. Just a curated, ever-improving wiki the agent maintains.

MindMemory implements that loop in the smallest possible form, plus measurement, plus a self-maintenance architecture that runs continuously rather than at session bookends.

MIT licensed. Repository, install instructions, full findings, and open questions: github.com/CoMindLab-ai/mind-memory